

Universidad Autónoma de Sinaloa
Facultad de Informática
Culiacán



Gestión de Seguridad y Desempeño de Bases de Datos

Alumno: Zamora Serrano Irvin Donovan

Profesor: Fidel Bojórquez Solis

Grado: 4 **Grupo:** 3

Práctica 1: Protección y Privacidad de Datos

Fecha de entrega: 23 de octubre de 2026

Actividad 1 - Clasificación e Identificación de Datos

1. Introducción

En esta práctica se documenta el despliegue de un entorno de base de datos relacional utilizando **Docker** y **PostgreSQL 16**. El propósito principal es estructurar y poblar una base de datos con Información Personal Identificable (PII) para su posterior análisis, clasificación y aplicación de estrategias de protección de datos personales.

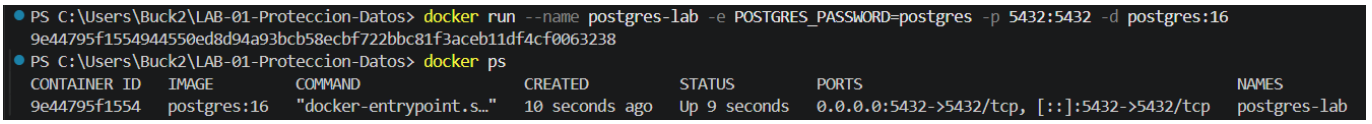
---

2. Desarrollo y Evidencias Visuales

Paso 1: Creación e Inicio del Contenedor PostgreSQL

Se ejecuta el contenedor `postgres-lab` mapeando el puerto `5432` y asignando las variables de entorno necesarias. Después se verifica con `docker ps` que el contenedor quedó activo (estado `Up`).

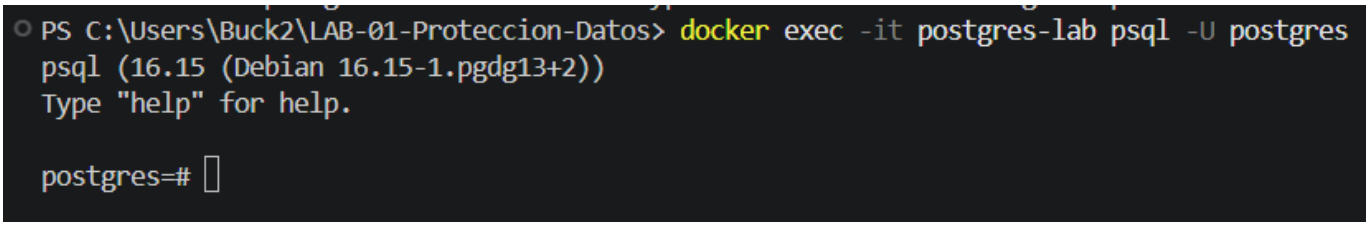
```
docker run --name postgres-lab -e POSTGRES_PASSWORD=postgres -p 5432:5432 -d
postgres:16
docker ps
```



Paso 2: Conexión Interactiva a PostgreSQL (`psql`)

Acceso a la consola de la base de datos dentro del contenedor. La versión mostrada (16.15) confirma que se está usando PostgreSQL 16.

```
docker exec -it postgres-lab psql -U postgres
```



Verificación completa del flujo de arranque (contenedor, estado y acceso a `psql`):

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker run --name postgres-lab -e POSTGRES_PASSWORD=postgres -p 5432:5432 -d postgres:16
9e44795f1554944550ed8d94a93bcb58ecbf722bbc81f3aceb11df4cf0063238
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS         PORTS                                                  NAMES
9e44795f1554   postgres:16   "docker-entrypoint.s…"   10 seconds ago   Up 9 seconds   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   postgres-lab
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

postgres=#
```

Paso 3: Creación de la Tabla `clientes`

Definición de la estructura de la tabla `clientes` que almacenará los registros con datos personales y sensibles:

```sql
CREATE TABLE clientes (
    id BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL,
    correo VARCHAR(150) NOT NULL,
    telefono VARCHAR(20),
    rfc VARCHAR(13),
    edad INTEGER,
    ciudad VARCHAR(80),
    codigo_postal VARCHAR(10),
    fecha_alta DATE DEFAULT CURRENT_DATE
);
```

```
postgres=# CREATE TABLE clientes (
    id BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL,
    correo VARCHAR(150) NOT NULL,
    telefono VARCHAR(20),
    rfc VARCHAR(13),
    edad INTEGER,
    ciudad VARCHAR(80),
    codigo_postal VARCHAR(10),
    fecha_alta DATE DEFAULT CURRENT_DATE
);
CREATE TABLE
postgres=#
```

Paso 4: Inspección de la Estructura de la Tabla

Comprobación de las columnas, tipos de datos y restricciones mediante el comando `\d clientes`:

```
\d clientes
```

```
postgres=# \d clientes
postgres=# \d clientes
                                  Table "public.clientes"
    Column     |          Type          | Collation | Nullable |              Default
---------------+------------------------+-----------+----------+-----------------------------------
 id            | bigint                 |           | not null | nextval('clientes_id_seq'::regclass)
 nombre        | character varying(120) |           | not null |
 correo        | character varying(150) |           | not null |
 telefono      | character varying(20)  |           |          |
 rfc           | character varying(13)  |           |          |
 edad          | integer                |           |          |
 ciudad        | character varying(80)  |           |          |
 codigo_postal | character varying(10)  |           |          |
 fecha_alta    | date                   |           |          | CURRENT_DATE
Indexes:
    "clientes_pkey" PRIMARY KEY, btree (id)

postgres=#
```

Paso 5: Inserción de Registros y Consulta General

Inserción de cinco registros de prueba con campos PII (datos ficticios, con dominio `example.test`) y consulta global mediante `SELECT * FROM clientes;`:

```
INSERT INTO clientes (nombre, correo, telefono, rfc, edad, ciudad, codigo_postal)
VALUES
('Ana López', 'ana.lopez@example.test', '6671234567', 'LOAA850315ABC', 41,
'Culiacán', '80000'),
('Carlos Pérez', 'carlos.perez@example.test', '6672345678', 'PECA900720DEF', 35,
'Culiacán', '80100'),
('María García', 'maria.garcia@example.test', '6673456789', 'GAMA780912GHI', 47,
'Mazatlán', '82000'),
('Luis Torres', 'luis.torres@example.test', '6674567890', 'TOLU920201JKL', 34,
'Los Mochis', '81200'),
('Laura Sánchez', 'laura.sanchez@example.test', '6675678901', 'SALA880525MNO', 38,
'Culiacán', '80200');

SELECT * FROM clientes;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=# INSERT INTO clientes (nombre, correo, telefono, rfc, edad, ciudad, codigo_postal)
VALUES
('Ana López', 'ana.lopez@example.test', '6671234567', 'LOAA850315ABC', 41, 'Culiacán', '80000'),
('Carlos Pérez', 'carlos.perez@example.test', '6672345678', 'PECA900720DEF', 35, 'Culiacán', '80100'),
('María García', 'maria.garcia@example.test', '6673456789', 'GAMA780912GHI', 47, 'Mazatlán', '82000'),
('Luis Torres', 'luis.torres@example.test', '6674567890', 'TOLU920201JKL', 34, 'Los Mochis', '81200'),
('Laura Sánchez', 'laura.sanchez@example.test', '6675678901', 'SALA880525MNO', 38, 'Culiacán', '80200');
INSERT 0 5
postgres=# SELECT * FROM clientes;
 id |    nombre     |          correo           |  telefono  |      rfc      | edad |   ciudad   | codigo_postal | fecha_alta
----+---------------+---------------------------+------------+---------------+------+------------+---------------+------------
  1 | Ana López     | ana.lopez@example.test    | 6671234567 | LOAA850315ABC |   41 | Culiacán   | 80000         | 2026-10-24
  2 | Carlos Pérez  | carlos.perez@example.test | 6672345678 | PECA900720DEF |   35 | Culiacán   | 80100         | 2026-10-24
  3 | María García  | maria.garcia@example.test | 6673456789 | GAMA780912GHI |   47 | Mazatlán   | 82000         | 2026-10-24
  4 | Luis Torres   | luis.torres@example.test  | 6674567890 | TOLU920201JKL |   34 | Los Mochis | 81200         | 2026-10-24
  5 | Laura Sánchez | laura.sanchez@example.test | 6675678901 | SALA880525MNO |   38 | Culiacán   | 80200         | 2026-10-24
(5 rows)
```

> **Nota:** la columna `fecha_alta` muestra `2026-10-24` porque el contenedor usa la hora UTC del servidor, mientras que la práctica se realizó el 23 de octubre en hora local.

---

3. Matriz de Clasificación de Datos

3.1 Escala de sensibilidad utilizada

Nivel	Criterio
Muy Alta	Identifica de forma única a la persona ante una autoridad (por ejemplo, el RFC). Su filtración tiene impacto fiscal o legal.
Alta	Identifica directamente a la persona (nombre, correo).
Media	Por sí solo no identifica, pero combinado con otros datos permite reidentificar (edad, código postal, teléfono).
Baja	Información general sin riesgo relevante de identificación (ciudad).
Interna	Dato técnico o de auditoría del sistema, sin información personal.

3.2 Clasificación por columna

Columna	Tipo de Dato	Clasificación	Sensibilidad	Acción Recomendada
`id`	`BIGINT`	Identificador Técnico	Pública / Interna	Sin cambio (Clave Primaria).
`nombre`	`VARCHAR(120)`	PII Directo	Alta	Enmascaramiento o Seudonimización.
`correo`	`VARCHAR(150)`	PII Directo	Alta	Enmascaramiento parcial (`a***@example.test`).
`telefono`	`VARCHAR(20)`	PII Directo	Media / Alta	Enmascaramiento / Randomización.
`rfc`	`VARCHAR(13)`	Identificador Fiscal	Muy Alta	Enmascaramiento / Anonimización.

Columna	Tipo de Dato	Clasificación	Sensibilidad	Acción Recomendada
`edad`	`INTEGER`	Cuasi-identificador	Media	Generalización por rangos de edad.
`ciudad`	`VARCHAR(80)`	Ubicación Geográfica	Baja / Media	Mantenimiento para análisis demográfico.
`codigo_postal`	`VARCHAR(10)`	Ubicación Geográfica	Media	Supresión parcial de dígitos finales.
`fecha_alta`	`DATE`	Auditoría Interna	Interna	Sin cambio.

4. Conclusión

Se verificó la correcta creación de la infraestructura (contenedor `postgres-lab` con PostgreSQL 16) y la inserción de datos de prueba en la tabla `clientes`. Las evidencias confirman el funcionamiento de Docker y PostgreSQL, y la matriz de clasificación identifica qué columnas contienen PII directo, cuasi-identificadores y datos técnicos, sentando las bases para aplicar los métodos de enmascaramiento, seudonimización y anonimización en las siguientes actividades.